



UNSW
SYDNEY

COMP4141

Theory of Computation

Lecture 13–14: **NP**-Completeness

Outline

Satisfiability of 4-CNF Formulae

Minimum Vertex Cover

Maximum Clique

Hamiltonian Path

The Travelling Salesman Problem

Subset Sum

Integer Linear Programming

4SAT

4-CNF: Conjunctive normal form with at most 4 literals per clause.

4SAT

Input: A boolean formula φ in 4-CNF (n variables, m clauses)

Question: Is φ satisfiable?

4SAT

Theorem

4SAT is **NP**-complete.

4SAT

Theorem

4SAT is **NP-complete**.

Lemma

4SAT $\in$ **NP**.

- Certificate: An assignment of variables to true or false
- Verifier:
 - Check that each of the m clauses has at least one variable that will be assigned true
- Running time: Checking each clause takes $O(4n)$ time. There are m clauses, so total time is $O(mn) \in \mathbf{TIME}(|\varphi|^2)$

4SAT

Lemma

4SAT is **NP-hard**.

Proof by reduction from 3SAT.

Consider the function $f : 3\text{-CNF} \rightarrow 4\text{-CNF}$ given by $f(\varphi) = \varphi$.

- A formula φ in 3-CNF, is also a formula in 4-CNF.
- f is computable in polynomial time.
- If $\varphi \in 3\text{SAT}$ then $f(\varphi)$ is satisfiable, so $f(\varphi) \in 4\text{SAT}$.
- If $\varphi \notin 3\text{SAT}$ then $f(\varphi)$ is not satisfiable, so $f(\varphi) \notin 4\text{SAT}$.

So $3\text{SAT} \leq_P 4\text{SAT}$. So 4SAT is **NP-hard**.

Outline

Satisfiability of 4-CNF Formulae

Minimum Vertex Cover

Maximum Clique

Hamiltonian Path

The Travelling Salesman Problem

Subset Sum

Integer Linear Programming

VERTEX COVER

A **vertex cover** is a subset of vertices such that every edge has at least one endpoint in the set (every edge is *covered*).

VERTEX COVER

Input: An undirected graph G and an integer k

Question: Does G have a k node vertex cover?

VERTEX COVER

Theorem

VERTEX COVER *is* **NP**-complete.

VERTEX COVER

Theorem

VERTEX COVER is **NP-complete**.

Lemma

VERTEX COVER $\in$ **NP**.

- Certificate: A subset of vertices V'
- Verifier:
 - Check that $|V'| = k$
 - Check each edge of G has at least one endpoint in V' .
- Running time: Checking each edge is covered takes at most $O(|V'|) = O(k)$ time. There are $|E|$ clauses, so total time is $O(k|E|) \in \mathbf{TIME}(|G|^2)$

VERTEX COVER

Lemma

VERTEX COVER *is NP-hard.*

Proof by reduction from 3SAT.

Given a formula φ in 3-CNF, with n variables and m clauses, set $k = n + 2m$ and construct the following graph:

- n “variable gadgets” ($2n$ vertices, n edges)

VERTEX COVER

Lemma

VERTEX COVER is **NP-hard**.

Proof by reduction from 3SAT.

Given a formula φ in 3-CNF, with n variables and m clauses, set $k = n + 2m$ and construct the following graph:

- n “variable gadgets” ($2n$ vertices, n edges)



VERTEX COVER

Lemma

VERTEX COVER is **NP-hard**.

Proof by reduction from 3SAT.

Given a formula φ in 3-CNF, with n variables and m clauses, set $k = n + 2m$ and construct the following graph:

- n “variable gadgets” ($2n$ vertices, n edges)
- m “clause gadgets” ($3m$ vertices, $3m$ edges)



VERTEX COVER

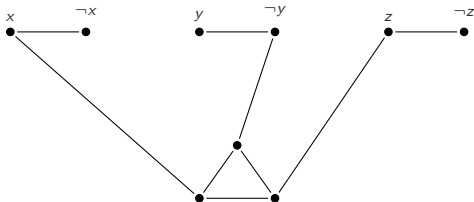
Lemma

VERTEX COVER is **NP-hard**.

Proof by reduction from 3SAT.

Given a formula φ in 3-CNF, with n variables and m clauses, set $k = n + 2m$ and construct the following graph:

- n “variable gadgets” ($2n$ vertices, n edges)
- m “clause gadgets” ($3m$ vertices, $3m$ edges)
- $3m$ connecting edges (e.g. for $(x \vee \neg y \vee z)$)



VERTEX COVER

G has $2n + 3m$ vertices and $6m + n$ edges, and is constructible in polynomial time, as is k .

It remains to show that $\varphi \in 3\text{SAT}$ if and only if $\langle G, k \rangle \in \text{VERTEX COVER}$.

VERTEX COVER: “if”

NB

To cover all the edges in the vertex and clause gadgets, a vertex cover must have at least $n + 2m$ vertices.

Therefore, if G has a vertex cover of size exactly $n + 2m$

- For each variable gadget, exactly one vertex is in the VC.
- For each clause gadget, exactly two vertices are in the VC.
 - $\Rightarrow$ 2 connecting edges are covered by vertices in the clause gadget
 - $\Rightarrow \geq 1$ connecting edge is covered by a vertex in a variable gadget

So if G has a vertex cover V' of size exactly $n + 2m$, we can assign a truth assignment as follows:

$$v(x) = \text{true if, and only if } x \in V'$$

From the above observations v will make at least one variable in each clause of φ true, so φ is satisfiable.

VERTEX COVER: “only if”

Suppose φ is satisfiable. Let v be a truth assignment which satisfies φ . Define V' as follows:

- If $v(x) = \text{true}$ then the vertex corresponding to x is in V'
- If $v(x) = \text{false}$ then the vertex corresponding to $\neg x$ is in V'
- For each clause there is at least one literal which is true: the vertices in the clause gadget which are not connected to the corresponding vertex are in V' .

We observe:

- V' has $n + 2m$ vertices.
- The edge in each variable gadget is covered.
- All edges in each clause gadget is covered.
- All connecting edges are covered.

Therefore, V' is a vertex cover of size $k = n + 2m$.

Outline

Satisfiability of 4-CNF Formulae

Minimum Vertex Cover

Maximum Clique

Hamiltonian Path

The Travelling Salesman Problem

Subset Sum

Integer Linear Programming

CLIQUE

CLIQUE

Input: An undirected graph G and an integer k

Question: Does G have a clique of size k ?

NB

A k -clique in an undirected graph is a set of k vertices such that there is an edge between each pair.

CLIQUE

Theorem

CLIQUE is **NP**-complete.

Proof covered in next tutorial:

Hint

What is the connection between a vertex cover and a clique?

Outline

Satisfiability of 4-CNF Formulae

Minimum Vertex Cover

Maximum Clique

Hamiltonian Path

The Travelling Salesman Problem

Subset Sum

Integer Linear Programming

HAMPATH

HAMPATH

Input: An undirected graph G

Question: Is there a path in G that visits every vertex exactly once?

HAMPATH

Theorem

HAMPATH is **NP**-complete.

HAMPATH

Theorem

HAMPATH is **NP**-complete.

Lemma

HAMPATH $\in$ **NP**.

Proof: Tutorial

HAMPATH

Lemma

HAMPATH *is* **NP-hard**.

HAMPATH

Lemma

DIRECTED HAMPATH is **NP-hard**.

Proof by reduction from 3SAT.

Given a formula φ in 3-CNF, with n variables and m clauses, construct the following **directed graph**:

- n “variable gadgets” $((4 + 2m)n$ vertices, $(4m + 8)n$ edges)

HAMPATH

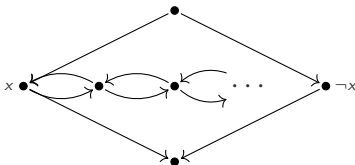
Lemma

DIRECTED HAMPATH is **NP-hard**.

Proof by reduction from 3SAT.

Given a formula φ in 3-CNF, with n variables and m clauses, construct the following **directed graph**:

- n “variable gadgets” $((4 + 2m)n$ vertices, $(4m + 8)n$ edges)



HAMPATH

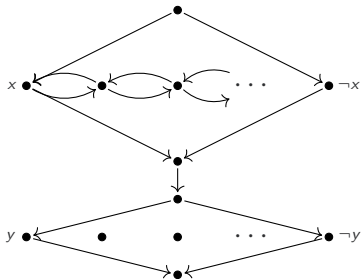
Lemma

DIRECTED HAMPATH is **NP-hard**.

Proof by reduction from 3SAT.

Given a formula φ in 3-CNF, with n variables and m clauses, construct the following **directed graph**:

- n “variable gadgets” $((4 + 2m)n$ vertices, $(4m + 8)n$ edges)



HAMPATH

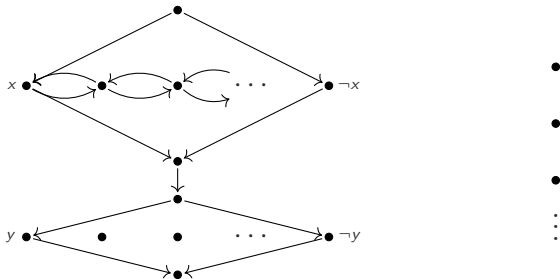
Lemma

DIRECTED HAMPATH is **NP-hard**.

Proof by reduction from 3SAT.

Given a formula φ in 3-CNF, with n variables and m clauses, construct the following **directed graph**:

- n “variable gadgets” $((4 + 2m)n$ vertices, $(4m + 8)n$ edges)
- m “clause vertices”



HAMPATH

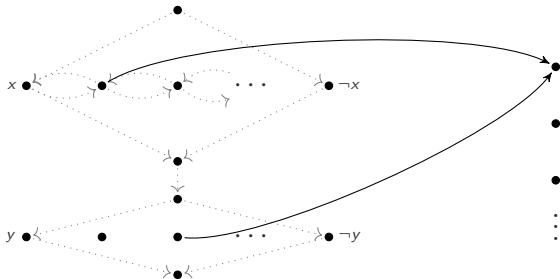
Lemma

DIRECTED HAMPATH is **NP-hard**.

Proof by reduction from 3SAT.

Given a formula φ in 3-CNF, with n variables and m clauses, construct the following **directed graph**:

- n “variable gadgets” $((4 + 2m)n$ vertices, $(4m + 8)n$ edges)
- m “clause vertices”
- $6m$ connecting edges (e.g. for $(x \vee \neg y)$)



HAMPATH

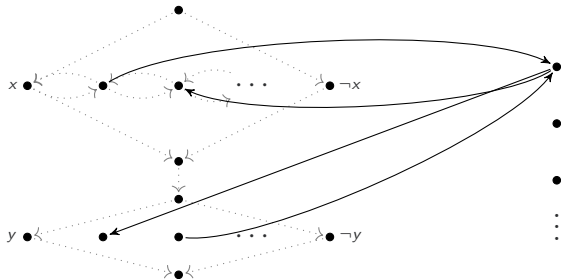
Lemma

DIRECTED HAMPATH is **NP-hard**.

Proof by reduction from 3SAT.

Given a formula φ in 3-CNF, with n variables and m clauses, construct the following **directed graph**:

- n “variable gadgets” $((4 + 2m)n$ vertices, $(4m + 8)n$ edges)
- m “clause vertices”
- $6m$ connecting edges (e.g. for $(x \vee \neg y)$)



Directed HAMPATH

The directed graph can be constructed in polynomial time. Now we show that the directed graph has a (directed) Hamiltonian path if and only if φ is satisfiable.

Directed HAMPATH: “only if”

Some observations:

- A Hamiltonian path either visits x before $\neg x$ (zig-zag) or $\neg x$ before x (zag-zig) and then goes to the next diamond
- A Hamiltonian path can only visit a clause where x occurs positively (negatively) on a zig-zag (zag-zig)
- When visiting a clause vertex, a Hamiltonian path must return to the same diamond gadget which it left

So setting $v(x) = \text{true}$ if a Hamiltonian zig-zags through the gadget corresponding to x , and false otherwise will give a truth assignment that satisfies φ .

Directed HAMPATH: “if”

Conversely, if φ has a satisfying assignment v , then we have the following Hamiltonian path:

- If $v(x) = \text{true}$ then zig-zag through the gadget corresponding to x , otherwise zag-zig
- If the truth value of x makes clause c true and the vertex corresponding to c has not been visited, then visit the vertex while zig-zagging/zag-zigging.

This directed path will clearly visit every vertex of the directed graph exactly once.

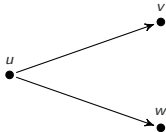
HAMPATH: Directed to undirected

It remains to show how we can remove the direction:

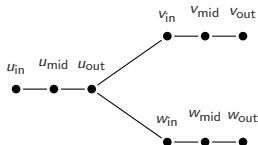
Lemma

$\text{DIRECTED_HAMPATH} \leq_P \text{HAMPATH}$

Replace



with



Outline

Satisfiability of 4-CNF Formulae

Minimum Vertex Cover

Maximum Clique

Hamiltonian Path

The Travelling Salesman Problem

Subset Sum

Integer Linear Programming

TRAVELLING SALESMAN

The TRAVELLING SALESMAN PROBLEM (TSP) is a “real-world” optimization problem.

TRAVELLING SALESMAN

Input: A set of cities and a table of “costs” for travelling between pairs of cities

Question: What is the least cost to visit all the cities?

TRAVELLING SALESMAN

As a decision problem:

TRAVELLING SALESMAN

Input: A set of cities, a table of “costs” for travelling between pairs of cities and a bound B

Question: Can you visit all cities with a cost $\leq B$?

NB

You can solve the optimization problem with an algorithm for the decision problem and binary search.

TRAVELLING SALESMAN

Theorem

TRAVELLING SALESMAN *is* **NP-complete**.

TRAVELLING SALESMAN

Theorem

TRAVELLING SALESMAN *is* **NP-complete**.

Lemma

TRAVELLING SALESMAN $\in$ **NP**.

- Certificate: An ordering of the cities: $c_1, c_2, \dots$
- Verifier:
 - Sum the cost of travelling from c_i to c_{i+1}
 - Check if the total cost is $\leq B$.
- Running time: Suppose there are n cities and the largest cost is C . Then we are adding at most n^2 numbers of size at most C , which will take time $O(n^2 L \log C)$ where L is the time to look up a single cost in the table. n , L , and $\log C$ are all bounded above by the input size of the table, so the running time is polynomial.

TRAVELLING SALESMAN

Lemma

TRAVELLING SALESMAN *is NP-hard.*

Proof by reduction from HAMPATH.

Given a graph $G = (V, E)$ define:

- The set of cities to be V
- The cost of travelling from u to v to be 1 if $\{u, v\} \in E$ and 2 otherwise
- The bound $B = |V| - 1$.

TRAVELLING SALESMAN

Clearly the instance of TRAVELLING SALESMAN can be constructed in polynomial time. It remains to show that it is a reduction.

- If G has a Hamiltonian path, then that path describes a tour that visits every city and costs exactly $|V| - 1$.
- Conversely, since every cost is at least 1, a tour that visits every city and costs at most $|V| - 1$ overall must:
 - Visit every vertex exactly once
 - Only travel between vertices connected with an edge

Therefore it will form a Hamiltonian path in the original graph.

Outline

Satisfiability of 4-CNF Formulae

Minimum Vertex Cover

Maximum Clique

Hamiltonian Path

The Travelling Salesman Problem

Subset Sum

Integer Linear Programming

SUBSET SUM

SUBSET SUM

Input: A set S of n integers: $\{s_1, s_2, \dots, s_n\}$ and an integer t

Question: Does there exist $S' \subseteq S$ such that $\sum_{s_i \in S'} s_i = t$?

SUBSET SUM

Theorem

SUBSET SUM *is* **NP-complete**.

SUBSET SUM

Theorem

SUBSET SUM is **NP-complete**.

Lemma

SUBSET SUM $\in$ **NP**.

- Certificate: A subset $S' \subseteq S$
- Verifier:
 - Sum the elements of S'
 - Check if the total is equal to t
- Running time: Suppose the largest magnitude in S is C . Then we are adding at most n numbers of size at most C , which will take time $O(n \log C)$, so the running time is polynomial.

SUBSET SUM

Lemma

SUBSET SUM is **NP-hard**.

Proof by reduction from VERTEX COVER.

Let $G = (V, E)$ be an undirected graph with n vertices and m edges, and k an integer.

Define the following set, S , of $n + m$ integers:

- For each vertex v_i : $y_i = 10^m + \sum_{j \text{ s.t. } v_i \in e_j} 10^j$
- For each edge e_j : $z_j = 10^j$

Set

$$t = k \times 10^m + 2 \sum_{j=0}^{m-1} 10^j.$$

SUBSET SUM

E.g. $E = \{\{v_0, v_1\}, \{v_0, v_2\}, \{v_2, v_3\}, \dots\}$

		e_{m-1}	$\cdots$	e_2	e_1	e_0
y_0	1	0	$\cdots$	0	1	1
y_1	1	0	$\cdots$	0	0	1
y_2	1	0	$\cdots$	1	1	0
y_3	1	0	$\cdots$	1	0	0
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
z_0						1
z_1					1	0
z_2				1	0	0
$\vdots$			$\vdots$	$\vdots$	$\vdots$	$\vdots$
z_{m-1}		1	$\cdots$	0	0	0
t	k	2	$\cdots$	2	2	2

SUBSET SUM

S is constructible in polynomial time, and t is computable in polynomial time. It remains to show that it is a reduction.

- If G has a vertex cover of size k then choose the following subset $S' \subseteq S$:
 - If v_i is in the vertex cover, then $y_i \in S'$
 - If e_j has only one endpoint in the cover, then $z_j \in S'$

It is easy to see that $\sum_{x \in S'} x = t$.

SUBSET SUM

S is constructible in polynomial time, and t is computable in polynomial time. It remains to show that it is a reduction.

- If G has a vertex cover of size k then choose the following subset $S' \subseteq S$:
 - If v_i is in the vertex cover, then $y_i \in S'$
 - If e_j has only one endpoint in the cover, then $z_j \in S'$

It is easy to see that $\sum_{x \in S'} x = t$.

- Conversely, if $S' \subseteq S$ is such that $\sum_{x \in S'} x = t$, then we observe:
 - S' contains exactly k numbers of the form y_i
 - For each $j \leq m - 1$ at least one of the two 10^j came from one of the y_i

Therefore, by setting $V' = \{v_i : y_i \in S'\}$ we see that V' is a vertex cover of size k .

Outline

Satisfiability of 4-CNF Formulae

Minimum Vertex Cover

Maximum Clique

Hamiltonian Path

The Travelling Salesman Problem

Subset Sum

Integer Linear Programming

INTEGER LINEAR PROGRAMMING

One of the most useful problem formulations:

INTEGER LINEAR PROGRAMMING

Input: A system of linear equations over the integers

Question: Does the system have a non-negative integer solution?

Often represented as an optimization problem:

- Maximize $c(\mathbf{x})$, subject to:
 - $A\mathbf{x} \leq \mathbf{b}$
 - $\mathbf{x} \geq \mathbf{0}$
 - $\mathbf{x} \in \mathbb{Z}^n$

NB

Inequalities can be expressed as equations using slack variables.

INTEGER LINEAR PROGRAMMING

Theorem

INTEGER LINEAR PROGRAMMING *is* **NP-complete**.

Not obviously in **NP**.

0-1 INTEGER LINEAR PROGRAMMING

A simplification:

0-1 INTEGER LINEAR PROGRAMMING

Input: A system of linear equations over the integers

Question: Does the system have a solution over $\{0, 1\}$?

0-1 INTEGER LINEAR PROGRAMMING

A simplification:

0-1 INTEGER LINEAR PROGRAMMING

Input: A system of linear equations over the integers

Question: Does the system have a solution over $\{0, 1\}$?

Lemma

$0-1 \text{ ILP} \leq_P \text{ILP}$

Proof.

Add the inequalities $x_i \leq 1$.



0-1 INTEGER LINEAR PROGRAMMING

Theorem

0-1 INTEGER LINEAR PROGRAMMING *is* **NP-complete**.

0-1 INTEGER LINEAR PROGRAMMING

Theorem

0-1 INTEGER LINEAR PROGRAMMING *is NP-complete.*

Lemma

0-1 INTEGER LINEAR PROGRAMMING $\in$ **NP**

- Certificate: Values (0 or 1) for each variable
- Verifier:
 - Evaluate each equation and check if it holds
- Running time: Each equation e can be evaluated in $O(|e|)$ time. For m equations, this will take $O(m|e|)$ time which is polynomial in the size of the input.

0-1 INTEGER LINEAR PROGRAMMING

Lemma

0-1 INTEGER LINEAR PROGRAMMING *is NP-hard*.

Proof by reduction from 3SAT, or VERTEX COVER, or
SUBSET SUM, ...

Given a graph G and an integer k , set up the following system of equations:

- For each vertex v_i , a variable x_i
- For each edge $e_j = \{v_i, v_k\}$ the inequality $x_i + x_k \geq 1$, and
- The equation $\sum x_i = k$.

– THE END –